# Runzo API 接口自动化测试框架

# 训练计划生成链路 - 实现方案

---

Requests + Pytest + Allure + MongoDB

版本 1.0 | 2026年4月

Python 3.9+ | Jenkins CI/CD

# 目录

一、项目架构概览

二、认证流程（设备登录）

三、计划生成链路 - 完整业务流程

四、异步轮询机制（MongoDB）

五、框架分层设计详解

六、API 层实现说明

七、测试用例设计策略

八、数据流与依赖链

九、配置管理方案（base + env 合并）

十、核心文件参考

# 一、项目架构概览

Runzo API 自动化测试框架采用分层架构设计，各层职责分离。修改一层不影响其他层，降低维护成本。

| 层级 | 目录 | 职责 | 变更频率 |
|---|---|---|---|
| 配置层 | config/ | 环境地址、数据库连接、公共请求头 | 低 |
| 公共工具层 | common/ | HTTP封装、日志、断言、缓存、轮询器 | 低 |
| API层 | api/ | 接口定义 - 每个业务模块一个类 | 低 |
| 测试数据层 | testdata/ | YAML参数化数据驱动 | 中 |
| Fixture层 | fixtures/ | Pytest夹具 - 认证、DB连接 | 中 |
| 用例层 | testcases/ | 业务验证逻辑 - 断言和校验 | 高 |

## 核心设计原则:

- API层只负责「怎么调」（路径、参数）。不包含任何断言。
- 用例层只负责「验证什么」（业务规则）。不关心HTTP细节。
- 数据层只负责「用什么值」（测试数据）。不包含逻辑。
- 接口路径变更时，只需修改API层的一行代码，所有用例无需改动。

# 二、认证流程（设备登录）

本系统使用「设备登录」认证方式。通过 deviceId
创建匿名用户，不需要用户名密码。这简化了自动化测试：每次测试会话自动创建新用户。

## 2.1 认证接口

| 属性 | 值 |
| --- | --- |
| 所属服务 | turing-user-center（独立于 turing-runner） |
| 请求方法 | POST |
| 接口路径 | /v1/auth/device-login |
| 请求体 | {"appName":"RunnerAI", "deviceId":"<UUID>", "appsflyerId":""} |
| 核心响应 | userId, accessToken, refreshToken, expiresIn |

## 2.2 turing-runner 必需的请求头

| 请求头 | 是否必填 | 说明 | 示例 |
| --- | --- | --- | --- |
| ts-user-id | 是 | 设备登录返回的用户ID | 95377812116000116 |
| ts-time-zone-id | 是 | 时区标识 | Asia/Shanghai |
| ts-country | 是 | 国家代码 | CN |
| Authorization | 是 | JWT令牌 Bearer {token} | Bearer eyJ0eXAi... |
| ts-run-unit | 否 | 距离单位（默认km） | km |
| lang | 否 | 语言偏好 | en |

## 2.3 框架中的实现

- api/auth_api.py — AuthAPI.device_login() 调用 user-center 服务
- fixtures/auth_fixtures.py — auth_user夹具(session级) 在会话开始时创建测试用户
- fixtures/auth_fixtures.py — runner_client夹具 将 userId + token 注入后续所有请求
- common/cache.py — userId 和 accessToken 存入全局缓存供跨模块使用

# 三、计划生成链路 - 完整业务流程

计划生成链路是整个系统的 P0 核心路径，涉及 2 个服务 + 1 个数据库，共 7 个步骤。

| 步骤 | 动作 | 接口/方式 | 关键数据 |
|---|---|---|---|
| 1 | 设备登录 | POST /v1/auth/device-login (user-center) | 提取: userId, accessToken |
| 2 | 设置请求头 | 框架内部处理 | ts-user-id, Authorization 等 |
| 3 | 生成计划（异步） | POST /runzo/plan/generate (turing-runner) | 发送: GeneratePlanDTO<br>触发异步生成 |
| 4 | 轮询MongoDB | MongoDB查询 runzo_training-plan | 条件: {createBy: userId}<br>等待: status=3 |
| 5 | 查询计划列表 | GET /runzo/plan/list | 提取: planId（核心依赖） |
| 6 | 查询周计划 | POST training-week-list | 验证: 每周训练安排 |
| 7 | 查询日计划 | GET untrained-dailies | 提取: dailyId（供训练使用） |

## 3.1 GeneratePlanDTO 请求体结构

| 字段 | 类型 | 必填 | 说明 |
|---|---|---|---|
| gender | int | 否 | 性别: 0=女, 1=男 |
| heightValue / heightUnit | str | 是 | 身高值和单位 如 175 / cm |
| weightValue / weightUnit | str | 是 | 体重值和单位 如 70 / kg |
| trainingGoal | str | 是 | 目标: 5km / 10km / half_marathon / full_marathon |
| planCompletionWeeks | int | 是 | 计划总周数 |
| trainingSchedule | array | 是 | 训练排期: ["MONDAY","WEDNESDAY","FRIDAY"] |
| lsdSchedule | str | 是 | 长距离跑安排日 如 SUNDAY |
| age | int | 是 | 用户年龄 |
| intensity | str | 否 | 训练强度: high / medium / low |
| trainingHistory | object | 是 | 训练历史: 配速、心率等 |

重要业务规则: 每个用户只能调用一次 plan/generate。因此异常测试的每条用例都需要创建独立的新用户。

# 四、异步轮询机制（MongoDB）

计划生成是异步操作。调用 /runzo/plan/generate 后，计划由后台 LLM 服务生成。我们通过轮询 MongoDB 来检查生成状态。

## 4.1 MongoDB 集合: runzo_training-plan

| status 值 | 含义 | 框架动作 |
|---|---|---|
| 1 | 生成中（进行中） | 继续轮询 |
| 3 | 生成成功 | 返回文档，停止轮询 |
| 4 | 生成失败 | 抛出 RuntimeError，停止轮询 |

## 4.2 轮询策略

- 查询条件: {createBy: user_id}
- 轮询间隔: 每 5 秒查询一次
- 超时时间: 最长等待 120 秒
- 成功条件: document.status == 3
- 失败条件: document.status == 4（抛出 RuntimeError）
- 超时条件: 已用时间 > 120秒（抛出 TimeoutError）

## 4.3 实现文件: common/waiter.py

- poll_until(condition_fn, timeout, interval) — 通用轮询函数，可复用于任何异步场景
- wait_for_plan_ready(mongo_db, user_id) — 计划专用封装，查询 MongoDB 并解释 status 字段

# 五、框架分层设计详解

## 5.1 HTTP 客户端 (common/http_client.py)

- 封装 requests.Session，统一管理 base_url、timeout、默认请求头
- 每个请求自动记录日志（方法、URL、状态码、耗时）
- 请求/响应详情自动附加为 Allure 报告附件
- 提供 set_headers() 和 set_token() 方法动态注入请求头

## 5.2 全局缓存 (common/cache.py)

解决接口上下游依赖问题。线程安全的单例模式，存储全局键值对。

| 缓存Key | 数据来源 | 使用方 |
|---|---|---|
| user_id | 设备登录响应 | 所有API请求头 (ts-user-id) |
| access_token | 设备登录响应 | 所有API请求头 (Authorization) |
| plan_id | plan/list 响应 | 周计划、日计划、统计、结算等 |
| daily_id | untrained-dailies 响应 | 开始训练、训练建议等 |

## 5.3 统一断言 (common/assertion.py)

- assert_status_code — HTTP 状态码校验
- assert_json_path — JSONPath 字段值校验
- assert_response_time — 响应时间 SLA 校验
- assert_json_schema — JSON Schema 契约校验
- assert_db_record — MongoDB 文档字段校验

所有断言失败时自动将详细信息附加到 Allure 报告（包含完整响应体和错误上下文）。

# 六、API 层实现说明

API层管理 138 个接口，分布在 12 个业务模块中。每个模块一个 API 类，继承 BaseAPI。

| API 类 | 文件 | 接口数 | 核心方法 |
|---|---|---|---|
| AuthAPI | api/auth_api.py | 1 | device_login() |
| PlanAPI | api/plan_api.py | 32 | generate(), get_list(), training_week_list()... |
| WorkoutAPI | (待实现) | 7 | start(), end(), control()... |
| SettlementAPI | (待实现) | 47 | log_details(), running_economy()... |
| StatisticsAPI | (待实现) | 13 | scores(), distance_trend()... |
| ChatAPI | (待实现) | 6 | stream_chat(), create_session()... |

## API 层设计规范:

- 每个方法对应一个接口端点
- 方法只负责组装参数和调用HTTP方法 —— 不包含任何断言
- 所有方法用 @allure.step 装饰，确保报告可追溯
- docstring 描述: 用途、参数、响应格式、示例
- PREFIX 类变量定义模块的 URL 路径前缀

# 七、测试用例设计策略

## 7.1 用例优先级体系

| 优先级 | 范围 | 触发时机 | 示例 |
|---|---|---|---|
| P0 / 冒烟 | 核心链路端到端 | 每次部署后 | 计划生成完整链路 |
| P1 | 模块级正向路径 | 每日回归 | 计划列表查询、周计划查询 |
| P2 | 异常/边界场景 | 每周回归 | 缺少必填字段 |
| P3 | 边缘罕见场景 | 发版回归 | 并发计划生成 |

## 7.2 异常用例的用户隔离机制

由于每个用户只能调用一次 plan/generate，异常测试采用「临时用户工厂」模式：

- create_temp_user fixture（function级） —— 每条用例创建独立新用户
- 每个临时用户有独立的 HttpClient 和请求头
- 用例结束后自动关闭临时 HttpClient
- 正向用例使用 session 级别的共享用户（auth_user）

## 7.3 数据驱动方法

测试数据在 testdata/plan/ 下的 YAML 文件中管理。每条数据包含 case_id、title、data（请求体）和 expected（断言条件）。DataLoader 加载 YAML 并提供给 @pytest.mark.parametrize，新增场景只需加一行 YAML 无需改代码。

# 八、数据流与依赖链

## 8.1 核心 ID 依赖链

整个系统围绕一条从 API 响应中提取的 ID 链运转：

| ID | 来源 | 使用方 | 存储方式 |
|---|---|---|---|
| userId | 设备登录 | 所有API请求头 | cache + fixture |
| planId | plan/list 或 MongoDB | 周计划、日计划、Block、统计、结算 | cache |
| dailyId | untrained-dailies | 开始训练、训练建议、日计划交换 | cache |
| sessionId | workout/start | 结束训练、暂停恢复、轨迹上传 | cache |
| logId | settlement/daily/logs | 所有结算分析接口 | cache |

## 8.2 双服务架构

| 服务 | Base URL (测试环境) | 用途 |
|---|---|---|
| turing-user-center | tsapiv1-test.shasoapp.com/turing-user-center | 用户认证 |
| turing-runner | tsapiv1-test.shasoapp.com/turing-runner | 核心业务 |

框架使用独立的 HttpClient 实例分别连接两个服务，在 conftest.py 中通过 user_center_client 和 runner_client 夹具管理。

# 九、配置管理方案（base + env 合并）

采用 base.yaml + {env}.yaml 分离模式，通过深度合并实现配置管理:

| 文件 | 内容 | 说明 |
| --- | --- | --- |
| config/base.yaml | 公共配置（timeout、headers、auth） | 所有环境共享，只写一次 |
| config/test.yaml | 测试环境差异（URL、MongoDB URI） | 只写与base不同的部分 |
| config/staging.yaml | 预发布环境差异 | 只写与base不同的部分 |

## 合并规则:

- 字典类型: 递归深度合并（env 的 key 覆盖 base 同名 key，其余保留）
- 简单值: env 直接覆盖 base
- 列表类型: env 直接替换（不做元素级合并）

示例: test.yaml 只定义了 mongodb.uri，base.yaml 定义了 mongodb.db_name，合并后两者都存在。

# 十、核心文件参考

| 文件路径 | 功能说明 |
| --- | --- |
| config/base.yaml | 基础配置: timeout、默认请求头、认证参数 |
| config/test.yaml | 测试环境: URL、MongoDB URI |
| config/staging.yaml | 预发布环境: URL、MongoDB URI |
| config/settings.py | 配置加载器: 深度合并 base + env |
| common/http_client.py | HTTP封装: 日志 + Allure附件自动集成 |
| common/logger.py | 日志管理: loguru 按天轮转 |
| common/cache.py | 全局缓存: 线程安全单例，解决接口关联 |
| common/waiter.py | 异步轮询: poll_until() + wait_for_plan_ready() |
| common/assertion.py | 统一断言: 状态码/JSONPath/Schema/DB/响应时间 |
| common/db_handler.py | MongoDB封装: URI连接 + CRUD操作 |
| common/data_loader.py | 数据加载器: YAML/JSON 参数化支持 |
| api/base_api.py | API基类: 持有 HttpClient 引用 |
| api/auth_api.py | 认证接口: 设备登录(user-center服务) |
| api/plan_api.py | 计划接口: 32个端点全部封装 |
| fixtures/auth_fixtures.py | 认证夹具: auth_user + runner_client |
| fixtures/db_fixtures.py | 数据库夹具: mongo_db + 数据清理 |
| testcases/conftest.py | 中央夹具: 客户端、API对象、导入 |
| testcases/test_plan/test_plan_generate.py | 计划生成: E2E链路 + 参数化异常测试 |
| testdata/plan/generate_plan.yaml | 计划生成: 参数化测试数据 |

## 运行命令:

```
pytest testcases/test_plan/ --env=test -v --alluredir=reports/allure-results
pytest testcases/test_plan/ --env=test -m smoke -v
```